RR Maxx

Nossa Equipe



Ismael Wecsley



Laiza Tavares



Mayara Aguiar



Paulo Pelicer



Victória Conceição

Contexto e Problema Identificado nas Oficinas

Desafios na gestão operacional

01



Controles manuais

Planilhas • Cadernos.

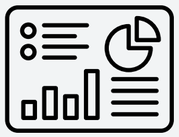
02



Serviços e garantias

Dificuldade de acompanhamento

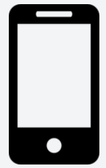
03



Falta de indicadores

Dados financeiros limitados

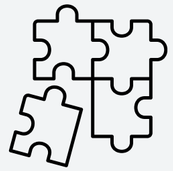
04



Comunicação com clientes

Sem notificações automáticas

05



Informações dispersas

Dados não centralizados

Como a Solução Apoia a RR Max Oficina

Benefícios da solução no dia a dia da oficina

01

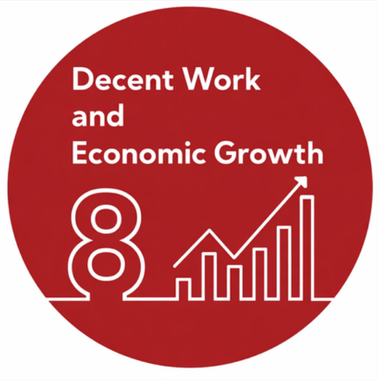
Facilidade no uso do sistema

Uso simples e prático no dia a dia da oficina.

02

Informações confiáveis

Registros corretos para apoiar decisões.



03

Organização dos dados

Clientes, veículos e serviços em um único sistema.

Demonstração da Solução (POC)

Execução prática das principais funcionalidades do sistema

Arquitetura: Clean Architecture nas Notificações

Refatoração de um bloco do backend para separar regra de negócio de framework

01

Antes

Uma classe de 436 linhas misturando regra de negócio, scheduler, banco, texto das mensagens e HTTP.

02

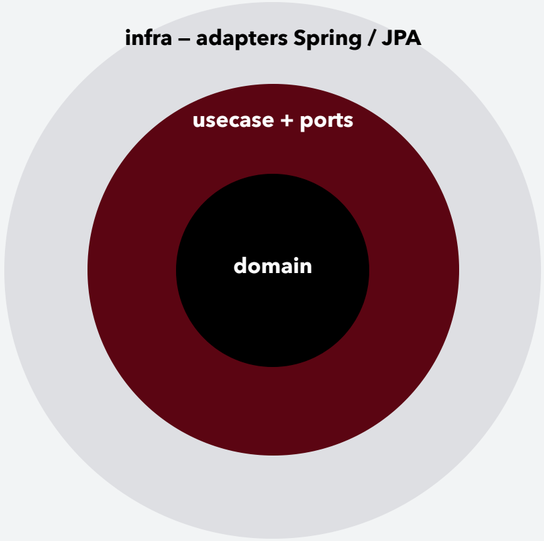
Depois

A regra fica no centro e conhece apenas **interfaces**. Spring, JPA e scheduler ficam na borda.

03

Ganho

Trocar o gateway (ex. Twilio) sem tocar na regra e testar a regra **sem subir o Spring**.



domain/ Telefone · MensagemNotificacao · NovaNotificacao

usecase/ NotificarFinalizacaoOrdem · AgendarLembreteRevisao · **port/** EnviadorDeMensagem · AgendadorDeLembrete · NotificacaoGateway

infra/ LogEnviadorDeMensagem · TaskSchedulerAgendadorDeLembrete · NotificacaoJpaGateway · NotificacaoUseCaseConfig

As dependências apontam sempre para dentro

Antes e Depois

O que a classe de regra precisa conhecer

01 Antes – NotificacaoService

Service/NotificacaoService.java — linhas 38–54

```java
@Service
public class NotificacaoService {

    private static final Logger log =
LoggerFactory.getLogger(NotificacaoService.class);
    private static final DateTimeFormatter DATA_PT_BR =
DateTimeFormatter.ofPattern("dd/MM/yyyy");
    private static final String ASSUNTO_FINALIZACAO = "Serviço
finalizado";
    private static final String ASSUNTO_REVISAO = "Lembrete de
revisão preventiva";

    private final TaskScheduler taskScheduler;
    private final NotificacaoRepository notificacaoRepository;
    private final Map<Long, ScheduledFuture<?>> lembretesAgendados =
new ConcurrentHashMap<>();
    private final Set<Long> lembretesProcessados =
ConcurrentHashMap.newKeySet();

    public NotificacaoService(TaskScheduler taskScheduler,
NotificacaoRepository notificacaoRepository) {
        this.taskScheduler = taskScheduler;
        this.notificacaoRepository = notificacaoRepository;
    }
```

Depende de `TaskScheduler` e `JpaRepository` e guarda estado do scheduler em mapas. Não dá para testar sem subir tudo.

02 Depois – AgendarLembreteRevisaoUseCase

notificacao/usecase/AgendarLembreteRevisaoUseCase.java — linhas 21–40

```java
public class AgendarLembreteRevisaoUseCase {

    public static final int DIAS_ANTECEDENCIA = 7;

    private final EnviadorDeMensagem enviador;
    private final NotificacaoGateway gateway;
    private final AgendadorDeLembrete agendador;
    private final Clock clock;

    public AgendarLembreteRevisaoUseCase(EnviadorDeMensagem
enviador,
                                         NotificacaoGateway gateway,
                                         AgendadorDeLembrete
agendador,
                                         Clock clock) {
        this.enviador = enviador;
        this.gateway = gateway;
        this.agendador = agendador;
        this.clock = clock;
    }

    public ResultadoNotificacao executar(OrdemParaNotificar ordem) {
```

Só interfaces (*ports*) e um `Clock`. Nenhuma anotação nem import de Spring – é uma classe Java comum.

A Regra de Negócio Fica Legível

Lembrete de revisão preventiva 7 dias antes da data prevista

notificacao/usecase/AgendarLembreteRevisaoUseCase.java — linhas 41–65

```
41          if (ordem.dataProximaRevisao() == null) {
42              return ResultadoNotificacao.naoEnviada("Data de revisão preventiva não informada.");
43          }
44
45          Long ordemId = ordem.id();
46          if (ordemId != null && agendador.foiProcessado(ordemId)) {
47              return ResultadoNotificacao.naoEnviada("Lembrete de revisão já processado nesta execução.");
48          }
49
50          LocalDateTime dataEnvio = ordem.dataProximaRevisao().minusDays(DIAS_ANTECEDENCIA);
51          LocalDateTime agora = LocalDateTime.now(clock);
52
53          if (!dataEnvio.isAfter(agora)) {
54              return enviarAgora(ordem, agora);
55          }
56
57          if (ordemId != null && agendador.estaAgendado(ordemId)) {
58              return ResultadoNotificacao.agendada(dataEnvio, "Lembrete de revisão já estava agendado.");
59          }
60
61          Long notificacaoId = registrarOuAtualizarPendente(ordem, dataEnvio);
62          agendador.agendar(ordemId, dataEnvio, () -> enviarAgendado(ordem, notificacaoId));
63
64          return ResultadoNotificacao.agendada(dataEnvio, "Lembrete de revisão preventiva agendado.");
65      }
```

01 Sem data → sai

02 Já processado → sai

03 Dentro do prazo → envia agora

04 Já agendado → sai

05 Senão → registra pendente e agenda

Port + Adapter

Trocar o envio para WhatsApp / Twilio é criar uma classe

01 O contrato – usecase/port

notificacao/usecase/port/EnviadorDeMensagem.java — linhas 7–16

```java
public interface EnviadorDeMensagem {

    /**
     * @param telefone destinatário já normalizado (somente dígitos)
     * @param mensagem texto a enviar
     * @param contexto descrição curta para rastreabilidade (ex.: "finalização da OS 12")
     * @return {@code true} se o envio foi aceito pelo gateway
     */
    boolean enviar(String telefone, String mensagem, String contexto);
}
```

02 A implementação atual – infra

notificacao/infra/LogEnviadorDeMensagem.java — linhas 13–24

```java
@Component
public class LogEnviadorDeMensagem implements EnviadorDeMensagem {

    private static final Logger log = LoggerFactory.getLogger(LogEnviadorDeMensagem.class);

    @Override
    public boolean enviar(String telefone, String mensagem, String contexto) {
        log.info("Notificação de {} registrada (envio externo desativado). Para: {} | Texto: {}",
                contexto, telefone, mensagem);
        return true;
    }
}
```

→

O caso de uso chama `enviador.enviar(...)` e não sabe quem está do outro lado

`TwilioEnviadorDeMensagem implements EnviadorDeMensagem` + `@Primary` – e nenhuma linha da regra muda.

Resultado: Regra Testada sem Spring e sem Banco

Primeiro teste unitário do projeto

test/notificacao/usecase/AgendarLembreteRevisaoUseCaseTest.java — linhas 36–58

```
36      @BeforeEach
37      void setUp() {
38          enviador = new EnviadorFake();
39          gateway = new GatewayFake();
40          agendador = new AgendadorFake();
41          Clock clock = Clock.fixed(AGORA.toInstant(ZoneOffset.UTC), ZoneId.of("UTC"));
42          useCase = new AgendarLembreteRevisaoUseCase(enviador, gateway, agendador, clock);
43      }
44
45      @Test
46      void revisaoDistanteAgendaLembreteSeteDiasAntes() {
47          OrdemParaNotificar ordem = ordem(AGORA.plusDays(30));
48
49          ResultadoNotificacao resultado = useCase.executar(ordem);
50
51          assertTrue(resultado.agendada());
52          assertFalse(resultado.enviada());
53          assertEquals(AGORA.plusDays(23), resultado.dataAgendamento());
54          assertEquals(AGORA.plusDays(23), agendador.quando);
55          assertEquals(1, gateway.registradas.size());
56          assertEquals(StatusNotificacao.pendente, gateway.registradas.get(0).status());
57          assertTrue(enviador.enviadas.isEmpty());
58      }
```

6 testes de regra

22 ms para rodar todos

0 Spring · banco · scheduler

Ports substituídos por *fakes* e `Clock.fixed`: o teste controla o tempo e verifica o que chegou em cada porta.

Agradecimentos

MINSAIT

SPTECH

Deloitte.

vivo

stefanini GROUP



AGRADECEMOS SUA ATENÇÃO

PRIME TECH
SOLUTIONS